Uke 0 - Utviklingsverktøy

IN1160

Dette dokumentet er en guide til hvordan dere kan sette opp et godt programmeringsmiljø og bli bedre til å skrive god kode. Dere står fritt til å gjøre dette slik dere vil, men dersom dere følger denne guiden vil dere lære om viktige kodekonvensjoner som er essentielt for å jobbe på større prosjekter og dersom man programmerer i jobbsammenheng.

Hjelpemidlene vi skal bruke fungerer automatisk, så dere vil altså få bedre kode uten å måtte gjøre mye selv. Dette dokumentet går igjennom hvordan dere setter opp programmeringsmiljøet den første gangen, samt hva programmene faktisk gjør mens de kjører automatisk. Dette oppsettet vil hjelpe dere både i dette faget og videre i deres programmeringskarriere.

Det ligger mye mer bak hva god kodebetyr enn kun hva et program gjør, hvor raskt det kjører og hvor mange linjer det inneholder. For å bli god til å skrive kode må man også gjøre koden lesbar, fleksibel og følge programmeringskonvensjoner. Slik blir det mulig å enkelt videreutvikle eksisterende kode, jobbe på større prosjekter med flere mennesker, skrive kode som raskt kan forstås av andre og forstå kode andre har skrevet. Dette er essentielle ferdigheter dersom man jobber med programmering, men vil også hjelpe dere å lære mer som studenter.

De fleste programmeringsspråk tillater mye forskjellig syntaks, men det betyr ikke at all syntaks er bra. For eksempel kan man ofte velge hvor mange mellomrom man bruker, hvilken rekkefølge man importerer i og hva variabelnavnene heter. Dette kan medføre at kode som gjør det samme kan se veldig forskjellig ut, som ved eksempelet under:

```
import  os ,   sys
from   math import   sqrt as square_root
def  f  ( x ) :
return square_root ( x  )+1
if __name__   ==    "__main__" :
result = f(  9 ) ;print (   f"Result of f(9) is: {result}"   )
```

Denne koden vil for de fleste bli regnet som veldig dårlig formatert. Koden under gjør akkurat det samme, men er godt formatert:

```
from math import sqrt as square_root
```

```
def f(x):
    return square_root(x) + 1


if __name__ == "__main__":
    result = f(9)
    print(f"Result of f(9) is: {result}")
```

Det finnes standarder for hvordan kode burde formateres med hensyn på mellomrom, importer og liknende. Heldigvis kan man bruke verktøy som automatisk fikser dette, så man slipper å lese om standardene og lære ting utenatt.

Oversikt

Her er en oversikt over noen praktiske former for verktøy:

- **Lintere**: er programmer som automatisk finner syntaks- eller formateringsfeil i koden mens den skrives. De vil kunne understreke eller highligte deler av koden som enten vil føre til feil, som at en variabel som ikke er definert blir brukt, eller er formatert galt, som at det er inkonsekvent antall mellomrom.
- **Formaterere**: kan automatisk formatere kode. Det vil si at de kan ta kode som er skrevet med inkonsekvent antall mellomrom eller plassering av kommaer og automatisk endre den til et standardformat. Formaterere kan enten bli konfigurert til å kjøre automatisk hver gang en fil blir lagret, eller ved å kjøre en kommando eller trykke på en snarvei.
- **IntelliSense**: gir kontekstbevisst hjelp mens du skriver kode, inkludert autofullføring av funksjoner og variabler, visning av dokumentasjon og funksjonssignaturer og mer.

Det finnes forskjellige implementasjoner av slike verktøy som varierer med teksteditor og programmeringsspråk. I denne guiden kommer vi til å presentere vårt anbefalt oppsett for Python i teksteditoren *Visual Studio Code*. Dette har blitt en utrolig populær teksteditor de siste årene og er derfor veldig nyttig å lære seg, men dere står fritt til å velge teksteditor selv.

Oppsett

Utvidelser

Start med å laste ned Visual Studio Code (VS Code). Dette er en populær teksteditor med stor støtte for forskjellige funksjoner som kan brukes med MacOS, Windows og Linux. Den har blant annet innebygd terminal og man kan kjøre Jupyter notebooks direkte i editoren (som vi skal se på senere i kurset). VS Code er fleksibelt og konfigurerbart gjennom innstillinger og hvilke extensions man bruker. Vi skal nå se på noen nyttige extensions for Python utvikling.

Start med å gå til *extensions* siden i venstre meny (eller shortcut `shift` + `cmd` / `ctrl` + `x`). Søk på og laste ned følgende:

- **Python**: Grunnleggende støtte for Python i VS Code. Det gir mulighet for kjøring av kode, feilmeldinger, debugger, og integrasjon med Jupyter og lintere.
- **Pylance**: Har blant annet en avansert IntelliSense som gjør det mulig med autocomplete og statisk analyse.
- **Ruff**: En ekstremt rask linter og formaterer implementert i Rust. Den er en reimplementasjon av linteren *flake8* og formattererene *black* og *isort*. Ruff har blitt veldig populært i det siste, grunnet at den er mye raskere enn tidligere implementasjoner og at den håndterer linting og formattering i én pakke.
- **Jupyter**: Gir funksjonalitet til å kjøre Jupyter notebooks (`.ipynb` filer) direkte i VS Code editoren. Vi skal se mer på Jupyter notebooks i `00-3jupyter_notebooks.ipynb`.
- **autoDocstring**: Vil automatisk lage en mal for docStrings i Python funksjoner som fyller ut argumenter, returneringer og exceptions.

Innstillinger

Klone dette git-repositoriet til datamaskinen deres om dere ikke allerede har gjort det. Det står beskrevet i `README.md` filen, men kort sagt trenger dere kun skrive `git clone https://github.uio.no/IN1160/v26.git` i terminalen.

Viktig: For at oppsettet i text editoren skal fungere, er det viktig at dere åpner hele mappen når dere jobber på prosjektet. Trykk altså `file -> Open folder` og velg hele mappen som heter `v26`. Dersom dere åpner undermapper (som `oblig2a`) eller enkeltfiler vil ikke oppsettet fungere ordentlig.

Det eneste som gjenstår nå er å passe på at utvidelsene er aktivert i innstillingene våre, som stort sett skjer helt automatisk. Dette prosjektet kommer med en fil `.vscode/settings.json` som inneholder innstillinger som passer på at utvidelsene er aktivert, men dere kan gå igjennom stegene i paragrafen under for å dobbeltsjekke dette samt å lære litt mer om innstillinger i VS Code.

Start med å gå til innstillinger i VS Code (tannhjulet nederst til venstre eller med snarvei `cmd / ctrl + ,`). Her er det to typer innstillinger, `User` og `Workspace`. Innstillingene i `Workspace` vil kun endre prosjektet (mappen) som er åpen når du endrer innstillingene, mens `User` vil endre for alle prosjekter. Innstillingene i `Workspace` vil bli lagret i en lokal fil `.vscode/settings.json` og vil overskrive `User` settings, som blir lagret i en global fil `settings.json` (som kan åpnes ved `shift + ctrl / cmd + p` og søk på `Open user settings`). Det går an å endre innstillingene ved å redigere JSON filen i tillegg til å endre dem fra innstillingsmenyen.

I innstillingene søk på `Default formatter` og velg `Ruff charliemarch.ruff`. Søk også `Ruff > Lint Enable` og sørg for at den er valgt. Deretter kan dere aktivere `Editor: Format on save`, som vil formatere koden automatisk hver gang den blir lagret.

Formattering og linting

For å formatere koden manuelt (dersom dere ikke bruker formatering ved lagring) trykk `shift + alt + f` for å formatere hele dokumentet. Dersom dere vil formatere en del av dokumentet kan dere fremheve tekst og trykke `ctrl / cmd + k + f` (hold nede `ctrl` / `cmd` knappen mens dere først trykker `k`, deretter `f`). For å formatere importer trykk `shift + alt / option + o`. Dersom koden har syntaks feil kan det være at formateringen ikke fungerer. Linteren vil kommentere både på syntaks feil (som gjør at koden ikke vil kjøre) og formattering (som handler om hvordan koden ser ut, men som ikke påvirker kompilatoren). Trykk `shift + cmd / ctrl + m` for å åpne alle linter issues.

Ruff vil ha en del meninger om hvordan koden skal se ut av design for å standardisere kodestiler, men det er mulig å endre noen innstillinger. Det kan dere gjøre enten ved å lage en fil `.ruff.toml` eller `pyproject.toml`, som vi bruker i dette prosjektet. Her kan dere for eksempel endre maks antall tegn på en linje (defaulten til Ruff er 88, men 120 er også vanlig). Vi har valgt å ignorere `F401` (ubrukt import) siden det er praktisk for Jupyter notebooks. Dersom dere vil at linteren skal overse en spesifikk linje, kan dere legge til en kommentar `# noqa <code>`, der `<code>` er koden for linteren oppgir. Dere kan se denne koden ved å holde over koden som linteren klager på.

Dere kan prøve å lage en Python fil (lagre den med `.py` endelse) med koden

under:

```
import  os ,   sys
from   math import   sqrt as square_root
def  f  ( x ) :
return square_root ( x  )+1
if __name__   ==    "__main__" :
result = f(  9 ) ;print (   f"Result of f(9) is: {result}"   )
```

Først kan dere trykke `shift` + `ctrl / cmd` + `m` for å se om linteren finner noen feil. Deretter kan dere formatere koden, enten ved å lagre (`ctrl / cmd` + `s`) eller ved å formatere manuelt `shift` + `alt / option` + `f`. Hurtigtasten `shift` + `alt / option` + `o` formaterer importene for seg. Endringene burde skje momentant. Tidligere formatterere kunne bruke et par sekunder på lengre filer, men Ruff er som regel superrrask.

God kodestil

For å skrive god kode er det vel så viktig å gjøre koden lesbar og vedlikeholdbar som å gjøre den effektiv og rask. Verktøyene vi har sett på over hjelper dere med mye, men det er også flere andre ting å tenke på. Punktlisten under går igjennom et par:

- **Gi gode navn**: Navn på variabler og funksjoner burde beskrive hva variablene og funksjonene gjør. Prøv generelt sett å unngå å gi navn slik som `x` og `f` dersom det er passende å gi mer spesifikke navn. Funksjonsnavn burde generelt sett inneholde et verb i imperativ form, slik som `process_data()` eller `calculate_accuracy()`. Variabelnavn kan være substantiv eller være modifisert med verb i fortid, slik som `accuracy` og `processed_data`.
- **Bruk snake_case i Python**: Det er mange måter å dele opp lange navn på. I Python er det konvensjon å bruke `snake_case`, der underscore (`_`) representerer mellomrom. Andre språk bruker andre konvensjoner, slik som camlCase i Java og JavaScript.
- **La én funksjon gjøre én ting**: Del opp programmet på en logisk og forståelig måte, både når det gjelder funksjoner, klasser og filer. Ikke la en funksjon gjøre mange forskjellige ting, del den opp i mindre hjelpefunksjoner.
- Generelt sett, prøv å gjøre koden lesbar og forståelig.

Oppsummering

Dette dokumentet har gått gjennom noen programmeringsverktøy som formaterer og lintere, hvordan sette opp et godt Python-miljø i VS Code, samt litt bakgrunn om hva verktøyene gjør og hvordan bruke VS Code.

Forhåpentligvis vil dere oppleve nytten av disse verktøyene mens dere gjør oppgaver og obliger i dette faget og andre fag. Det er mange mulige konfigurasjoner og verktøy utover det som er nevnt her. Dersom dere merker at dere foretrekker et annet setup kan dere justere dette underveis. Generativ kunstig intelligens kan være til stor hjelp for å finne ut hvilke spesifikke innstillinger man må endre for å få et spesifikt resultat.